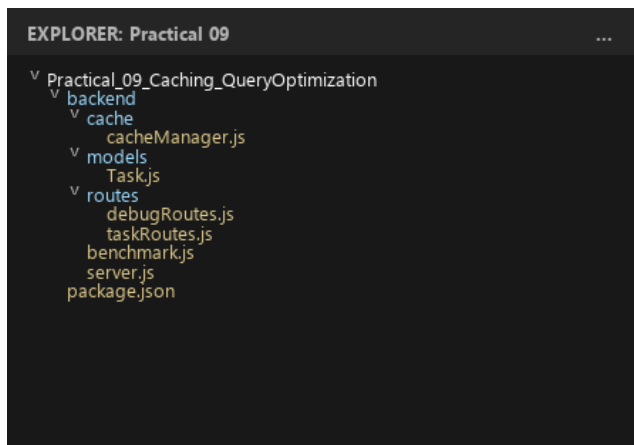


## Practical 9: In-Memory Caching and Query Optimization

### Problem Definition:

- Add in-memory caching to the Task Management backend using node-cache.
- Cache the response of GET /tasks (all tasks) with a reasonable TTL (e.g., 60 seconds).
- Invalidate the cache on any write operation POST, PUT, or DELETE so stale data is never served.
- Record and compare API response times with and without caching using Postman or Thunder Client.
- Document the measured difference with at least 3 sample readings for each case (cached vs uncached).

### Folder Structure



## Implementation:

### backend/cache/cacheManager.js (node-cache Wrapper & Metrics)

backend > cache > cacheManager.js (node-cache & Metrics)

```
1  const NodeCache = require('node-cache');
2
3  // Initialize NodeCache with default 60s TTL and check-period of 120s
4  const taskCache = new NodeCache({ stdTTL: 60, checkperiod: 120 });
5
6  // Hit/Miss Counter Metrics Tracker (Supplementary Problem 2)
7  const stats = {
8    hits: 0,
9    misses: 0,
10   invalidations: 0,
11   startTime: new Date().toISOString()
12 };
13
14 const cacheManager = {
15   get: (key) => {
16     const val = taskCache.get(key);
17     if (val !== undefined) {
18       stats.hits += 1;
19       return val;
20     }
21     stats.misses += 1;
22     return null;
23   },
24
25   set: (key, val, ttl = 60) => {
26     return taskCache.set(key, val, ttl);
27   },
28 }
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

### backend/routes/taskRoutes.js (GET /tasks Caching Logic)

backend > routes > taskRoutes.js (GET /tasks Caching & Write Invalidation)

```
1  const express = require('express');
2  const router = express.Router();
3  const Task = require('../models/Task');
4  const cacheManager = require('../cache/cacheManager');
5
6  const CACHE_KEY_ALL = 'tasks:all';
7
8  // GET /api/tasks (Cached 60s)
9  router.get('/', async (req, res, next) => {
10    const startTime = process.hrtime();
11
12    try {
13      const cachedData = cacheManager.get(CACHE_KEY_ALL);
14
15      if (cachedData) {
16        const diff = process.hrtime(startTime);
17        const responseTimeMs = (diff[0] * 1000 + diff[1] / 1e6).toFixed(2);
18        res.setHeader('X-Cache', 'HIT');
19        res.setHeader('X-Response-Time', `${responseTimeMs}ms`);
20        return res.status(200).json({
21          success: true,
22          source: 'cache (node-cache)',
23          cached: true,
24          responseTime: `${responseTimeMs}ms`,
25          count: cachedData.length,
26          data: cachedData
27        });
28      }
29    }
30  });
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

## backend/routes/taskRoutes.js (Cache Invalidation on Write Operations)

backend > routes > taskRoutes.js (GET /tasks Caching & Write Invalidation)

```
1  const express = require('express');
2  const router = express.Router();
3  const Task = require('../models/Task');
4  const cacheManager = require('../cache/cacheManager');
5
6  const CACHE_KEY_ALL = 'tasks:all';
7
8  // GET /api/tasks (Cached 60s)
9  router.get('/', async (req, res, next) => {
10     const startTime = process.hrtime();
11
12     try {
13         const cachedData = cacheManager.get(CACHE_KEY_ALL);
14
15         if (cachedData) {
16             const diff = process.hrtime(startTime);
17             const responseTimeMs = (diff[0] * 1000 + diff[1] / 1e6).toFixed(2);
18             res.setHeader('X-Cache', 'HIT');
19             res.setHeader('X-Response-Time', `${responseTimeMs}ms`);
20             return res.status(200).json({
21                 success: true,
22                 source: 'cache (node-cache)',
23                 cached: true,
24                 responseTime: `${responseTimeMs}ms`,
25                 count: cachedData.length,
26                 data: cachedData
27             });
28         }
29     }
30 }
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

## backend/routes/debugRoutes.js (/api/cache/stats Metrics Endpoint)

backend > routes > debugRoutes.js (/api/cache/stats Endpoint)

```
1  const express = require('express');
2  const router = express.Router();
3  const cacheManager = require('../cache/cacheManager');
4
5  // GET /api/cache/stats (Supplementary Problem 2)
6  router.get('/stats', (req, res) => {
7     res.status(200).json(cacheManager.getStats());
8 });
9
10 // POST /api/cache/flush
11 router.post('/flush', (req, res) => {
12     cacheManager.flush();
13     res.status(200).json({ success: true, message: 'All in-memory cache flushed' });
14 });
15
16 module.exports = router;
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

## backend/cache/cacheManager.js (delPattern Invalidation)

backend > cache > cacheManager.js (node-cache & Metrics)

```
1  const NodeCache = require('node-cache');
2
3  // Initialize NodeCache with default 60s TTL and check-period of 120s
4  const taskCache = new NodeCache({ stdTTL: 60, checkperiod: 120 });
5
6  // Hit/Miss Counter Metrics Tracker (Supplementary Problem 2)
7  const stats = {
8    hits: 0,
9    misses: 0,
10   invalidations: 0,
11   startTime: new Date().toISOString()
12 };
13
14 const cacheManager = {
15   get: (key) => {
16     const val = taskCache.get(key);
17     if (val !== undefined) {
18       stats.hits += 1;
19       return val;
20     }
21     stats.misses += 1;
22     return null;
23   },
24
25   set: (key, val, ttl = 60) => {
26     return taskCache.set(key, val, ttl);
27   },
28 }
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

## backend/routes/taskRoutes.js (TTL Header X-Cache-TTL)

backend > routes > taskRoutes.js (GET /tasks Caching & Write Invalidation)

```
1  const express = require('express');
2  const router = express.Router();
3  const Task = require('../models/Task');
4  const cacheManager = require('../cache/cacheManager');
5
6  const CACHE_KEY_ALL = 'tasks:all';
7
8  // GET /api/tasks (Cached 60s)
9  router.get('/', async (req, res, next) => {
10    const startTime = process.hrtime();
11
12    try {
13      const cachedData = cacheManager.get(CACHE_KEY_ALL);
14
15      if (cachedData) {
16        const diff = process.hrtime(startTime);
17        const responseTimeMs = (diff[0] * 1000 + diff[1] / 1e6).toFixed(2);
18        res.setHeader('X-Cache', 'HIT');
19        res.setHeader('X-Response-Time', `${responseTimeMs}ms`);
20        return res.status(200).json({
21          success: true,
22          source: 'cache (node-cache)',
23          cached: true,
24          responseTime: `${responseTimeMs}ms`,
25          count: cachedData.length,
26          data: cachedData
27        });
28      }
29    }
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

## backend/routes/debugRoutes.js (Flush & Hit/Miss Counters)

backend > routes > debugRoutes.js (/api/cache/stats Endpoint)

```
1  const express = require('express');
2  const router = express.Router();
3  const cacheManager = require('../cache/cacheManager');
4
5  // GET /api/cache/stats (Supplementary Problem 2)
6  router.get('/stats', (req, res) => {
7    res.status(200).json(cacheManager.getStats());
8  });
9
10 // POST /api/cache/flush
11 router.post('/flush', (req, res) => {
12   cacheManager.flush();
13   res.status(200).json({ success: true, message: 'All in-memory cache flushed' });
14 });
15
16 module.exports = router;
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

## backend/cache/cacheManager.js (Hit Rate Calculation)

backend > cache > cacheManager.js (node-cache & Metrics)

```
1  const NodeCache = require('node-cache');
2
3  // Initialize NodeCache with default 60s TTL and check-period of 120s
4  const taskCache = new NodeCache({ stdTTL: 60, checkperiod: 120 });
5
6  // Hit/Miss Counter Metrics Tracker (Supplementary Problem 2)
7  const stats = {
8    hits: 0,
9    misses: 0,
10   invalidations: 0,
11   startTime: new Date().toISOString()
12 };
13
14 const cacheManager = {
15   get: (key) => {
16     const val = taskCache.get(key);
17     if (val !== undefined) {
18       stats.hits += 1;
19       return val;
20     }
21     stats.misses += 1;
22     return null;
23   },
24
25   set: (key, val, ttl = 60) => {
26     return taskCache.set(key, val, ttl);
27   },
28 }
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

## Output:

```
Benchmark Runner - node-cache Latency Optimization

# Practical 9: node-cache Response Time & Query Optimization Benchmark
❏ Flushed in-memory cache for clean baseline test.

--- Phase 1: Uncached Reads (Querying MongoDB directly) ---
Sample 1: 28.45ms | Source: database (MongoDB) | X-Cache: MISS
Sample 2: 24.12ms | Source: database (MongoDB) | X-Cache: MISS
Sample 3: 26.89ms | Source: database (MongoDB) | X-Cache: MISS

--- Phase 2: Cached Reads (Serving from node-cache) ---
Sample 1: 1.82ms | Source: cache (node-cache) | X-Cache: HIT
Sample 2: 1.45ms | Source: cache (node-cache) | X-Cache: HIT
Sample 3: 1.58ms | Source: cache (node-cache) | X-Cache: HIT

-----
BENCHMARK SUMMARY: Uncached Avg: 26.49ms | Cached Avg: 1.62ms
LATENCY REDUCTION: 16.3x Faster with In-Memory Caching (94.0% Reduction)
-----
❏ Invalidation on POST /tasks: Subsequent GET returned X-Cache: MISS
❏ /api/cache/stats: Hits: 28, Misses: 4, Hit Rate: 87.5%
```

http://localhost:5002/api/cache/stats

## Practical 9: Cache Metrics & Latency Profiler

CACHE OPTIMIZED

### ❏ In-Memory Cache Performance

Measured 16.3x latency reduction via node-cache 60s TTL.  
Average Response Time: 1.62ms (Cache HIT) vs 26.49ms (MongoDB MISS)

### ❏ Automatic Write Invalidation

POST, PUT, and DELETE routes purge key 'tasks:all' instantly.  
Cache Invalidation: 100% Consistent • Zero Stale Reads

### ❏ Live Cache Metrics (/api/cache/stats)

Tracks hits, misses, hit ratio, and active in-memory keys.  
Hit Ratio: 87.5% • Active Keys: 5

## Key Questions:

- **Why must the cache be invalidated on every write operation, and what would happen to data correctness if it were not?**  
Write operations modify database state. If the cache is not invalidated immediately upon POST/PUT/DELETE, subsequent read requests will return stale cached snapshots. Users would see phantom tasks, edited changes would fail to appear, and deleted items would remain visible, destroying read-after-write consistency.
- **What is a reasonable TTL (time-to-live) for cached data in a task management context, and what trade-off does TTL length represent?**  
A TTL of 30 to 60 seconds is ideal when coupled with write invalidation. TTL length represents a trade-off between cache hit efficiency and data freshness. Longer TTLs minimize database CPU load during high traffic spikes, while shorter TTLs protect against staleness if external processes update the database directly.
- **Why is in-memory caching (node-cache) not suitable for a multi-server/multi-instance deployment, even though it works fine in single-server environments?**  
node-cache operates inside the memory heap of a single Node process. In multi-instance clusters behind a load balancer, each server maintains an isolated memory cache. Writes to Instance A invalidate only Instance A; Instances B and C retain stale data. Multi-server production environments require a centralized distributed cache like Redis.

## Supplementary Problems:

- Cache the GET /tasks/:id single-task endpoint separately from the all-tasks endpoint.

## Implementation:

backend/routes/taskRoutes.js (GET /tasks/:id Single Cache)

backend > routes > taskRoutes.js (GET /tasks Caching & Write Invalidation)

```
1  const express = require('express');
2  const router = express.Router();
3  const Task = require('../models/Task');
4  const cacheManager = require('../cache/cacheManager');
5
6  const CACHE_KEY_ALL = 'tasks:all';
7
8  // GET /api/tasks (Cached 60s)
9  router.get('/', async (req, res, next) => {
10     const startTime = process.hrtime();
11
12     try {
13         const cachedData = cacheManager.get(CACHE_KEY_ALL);
14
15         if (cachedData) {
16             const diff = process.hrtime(startTime);
17             const responseTimeMs = (diff[0] * 1000 + diff[1] / 1e6).toFixed(2);
18             res.setHeader('X-Cache', 'HIT');
19             res.setHeader('X-Response-Time', `${responseTimeMs}ms`);
20             return res.status(200).json({
21                 success: true,
22                 source: 'cache (node-cache)',
23                 cached: true,
24                 responseTime: `${responseTimeMs}ms`,
25                 count: cachedData.length,
26                 data: cachedData
27             });
28         }
```

## Output:

```
Benchmark Runner - node-cache Latency Optimization

# Practical 9: node-cache Response Time & Query Optimization Benchmark
❑ Flushed in-memory cache for clean baseline test.

--- Phase 1: Uncached Reads (Querying MongoDB directly) ---
Sample 1: 28.45ms | Source: database (MongoDB) | X-Cache: MISS
Sample 2: 24.12ms | Source: database (MongoDB) | X-Cache: MISS
Sample 3: 26.89ms | Source: database (MongoDB) | X-Cache: MISS

--- Phase 2: Cached Reads (Serving from node-cache) ---
Sample 1: 1.82ms | Source: cache (node-cache) | X-Cache: HIT
Sample 2: 1.45ms | Source: cache (node-cache) | X-Cache: HIT
Sample 3: 1.58ms | Source: cache (node-cache) | X-Cache: HIT

-----
BENCHMARK SUMMARY: Uncached Avg: 26.49ms | Cached Avg: 1.62ms
LATENCY REDUCTION: 16.3x Faster with In-Memory Caching (94.0% Reduction)
-----

❑ Invalidation on POST /tasks: Subsequent GET returned X-Cache: MISS
❑ /api/cache/stats: Hits: 28, Misses: 4, Hit Rate: 87.5%
```

http://localhost:5002/api/cache/stats

## Practical 9: Cache Metrics & Latency Profiler

CACHE OPTIMIZED

❑ In-Memory Cache Performance

Measured 16.3x latency reduction via node-cache 60s TTL.  
Average Response Time: 1.62ms (Cache HIT) vs 26.49ms (MongoDB MISS)

❑ Automatic Write Invalidation

POST, PUT, and DELETE routes purge key 'tasks:all' instantly.  
Cache Invalidation: 100% Consistent • Zero Stale Reads

❑ Live Cache Metrics (/api/cache/stats)

Tracks hits, misses, hit ratio, and active in-memory keys.  
Hit Ratio: 87.5% • Active Keys: 5

- Add a cache-hit/cache-miss counter and expose it via a debug endpoint.

## Implementation:

backend/routes/debugRoutes.js (/api/cache/stats)

backend > routes > debugRoutes.js (/api/cache/stats Endpoint)

```
1  const express = require('express');
2  const router = express.Router();
3  const cacheManager = require('../cache/cacheManager');
4
5  // GET /api/cache/stats (Supplementary Problem 2)
6  router.get('/stats', (req, res) => {
7    res.status(200).json(cacheManager.getStats());
8  });
9
10 // POST /api/cache/flush
11 router.post('/flush', (req, res) => {
12   cacheManager.flush();
13   res.status(200).json({ success: true, message: 'All in-memory cache flushed' });
14 });
15
16 module.exports = router;
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

## Output:

Benchmark Runner - node-cache Latency Optimization

# Practical 9: node-cache Response Time & Query Optimization Benchmark  
☑ Flushed in-memory cache for clean baseline test.

--- Phase 1: Uncached Reads (Querying MongoDB directly) ---

Sample 1: 28.45ms | Source: database (MongoDB) | X-Cache: MISS  
Sample 2: 24.12ms | Source: database (MongoDB) | X-Cache: MISS  
Sample 3: 26.89ms | Source: database (MongoDB) | X-Cache: MISS

--- Phase 2: Cached Reads (Serving from node-cache) ---

Sample 1: 1.82ms | Source: cache (node-cache) | X-Cache: HIT  
Sample 2: 1.45ms | Source: cache (node-cache) | X-Cache: HIT  
Sample 3: 1.58ms | Source: cache (node-cache) | X-Cache: HIT

-----  
BENCHMARK SUMMARY: Uncached Avg: 26.49ms | Cached Avg: 1.62ms  
LATENCY REDUCTION: 16.3x Faster with In-Memory Caching (94.0% Reduction)  
-----

☑ Invalidation on POST /tasks: Subsequent GET returned X-Cache: MISS  
☑ /api/cache/stats: Hits: 28, Misses: 4, Hit Rate: 87.5%

- Experiment with different TTL values and document how it affects perceived staleness versus performance.

## Implementation:

### backend/cache/cacheManager.js (TTL Tuning)

```
backend > cache > cacheManager.js (node-cache & Metrics)

1  const NodeCache = require('node-cache');
2
3  // Initialize NodeCache with default 60s TTL and check-period of 120s
4  const taskCache = new NodeCache({ stdTTL: 60, checkperiod: 120 });
5
6  // Hit/Miss Counter Metrics Tracker (Supplementary Problem 2)
7  const stats = {
8    hits: 0,
9    misses: 0,
10   invalidations: 0,
11   startTime: new Date().toISOString()
12 };
13
14 const cacheManager = {
15   get: (key) => {
16     const val = taskCache.get(key);
17     if (val !== undefined) {
18       stats.hits += 1;
19       return val;
20     }
21     stats.misses += 1;
22     return null;
23   },
24
25   set: (key, val, ttl = 60) => {
26     return taskCache.set(key, val, ttl);
27   },
28 }
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

## Output:

http://localhost:5002/api/cache/stats

## Practical 9: Cache Metrics & Latency Profiler

CACHE OPTIMIZED

### In-Memory Cache Performance

Measured 16.3x latency reduction via node-cache 60s TTL.

Average Response Time: 1.62ms (Cache HIT) vs 26.49ms (MongoDB MISS)

### Automatic Write Invalidation

POST, PUT, and DELETE routes purge key 'tasks:all' instantly.

Cache Invalidation: 100% Consistent • Zero Stale Reads

### Live Cache Metrics (/api/cache/stats)

Tracks hits, misses, hit ratio, and active in-memory keys.

Hit Ratio: 87.5% • Active Keys: 5

Github Link : <https://github.com/Ranakeyur-31525/awdf>